

Lab 5 – Monitoring, Logging & Incident Detection

Name: Sicid Sukhra Abdirahman

ID:52215124928

Objective

The objective of this lab is to understand and apply centralized logging, security event monitoring, tamper-evident logging, incident detection, containment, and evidence integrity. The lab also demonstrates how multiple security events can be correlated to identify suspicious activities and support incident response.

Environment

The following tools and environment were used to complete this lab:

- Windows 11 with Git Bash for running commands.
- Docker Desktop for running LocalStack and containment containers.
- LocalStack for simulating AWS CloudWatch Logs.
- AWS CLI for creating and accessing log groups and log streams.
- grep and awk for log analysis.
- sha256sum for hash-chain creation and evidence integrity verification.
- iptables for blocking the suspicious IP address.

Session A – Monitoring & Logging

Task 1 – Generate Application Logs

In this task, I created an application log file containing authentication and data export events. The log included successful logins, failed login attempts, and a data export activity.

Commands

```
```bash
cat > auth.log <<'EOF'
2025-03-01T09:00:01 LOGIN_OK user=ahmad ip=10.0.0.5
2025-03-01T09:01:10 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:12 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:15 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:18 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:22 LOGIN_OK user=admin ip=203.0.113.9
2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB
EOF

cat auth.log
```
```

Result

The `auth.log` file was created successfully with seven security-related events.

The log showed one successful login from `10.0.0.5`, four failed login attempts from `203.0.113.9`, one successful login from the same suspicious IP address, and one 500 MB data export event.

Evidence

```
user@LAPTOP-SN02HB52 MINGW64 ~  
$ cat auth.log  
2025-03-01T09:00:01 LOGIN_OK user=ahmad ip=10.0.0.5  
2025-03-01T09:01:10 LOGIN_FAIL user=admin ip=203.0.113.9  
2025-03-01T09:01:12 LOGIN_FAIL user=admin ip=203.0.113.9  
2025-03-01T09:01:15 LOGIN_FAIL user=admin ip=203.0.113.9  
2025-03-01T09:01:18 LOGIN_FAIL user=admin ip=203.0.113.9  
2025-03-01T09:01:22 LOGIN_OK user=admin ip=203.0.113.9  
2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB  
  
user@LAPTOP-SN02HB52 MINGW64 ~  
$ |
```

Task 2 – Centralize Logs Using LocalStack

In this task, I used LocalStack and AWS CloudWatch Logs to centralize the application logs. I created a log group and log stream, and successfully uploaded and retrieved events from `auth.log`.

Commands

```
```bash
```

```
docker run -d --name localstack -p 4566:4566 localstack/localstack
```

```
EP='--endpoint-url=http://localhost:4566'
```

```
MSYS_NO_PATHCONV=1 aws $EP logs create-log-group \
--log-group-name /ccse/app
```

```
MSYS_NO_PATHCONV=1 aws $EP logs create-log-stream \
--log-group-name /ccse/app \
--log-stream-name auth
```

```
TS=$(date +%s000)
```

```
while IFS= read -r line; do
 MSYS_NO_PATHCONV=1 aws $EP logs put-log-events \
 --log-group-name /ccse/app \
 --log-stream-name auth \
 --log-events timestamp=$TS,message="$line" >/dev/null
 TS=$((TS+1000))
done
```

```
done < auth.log
```

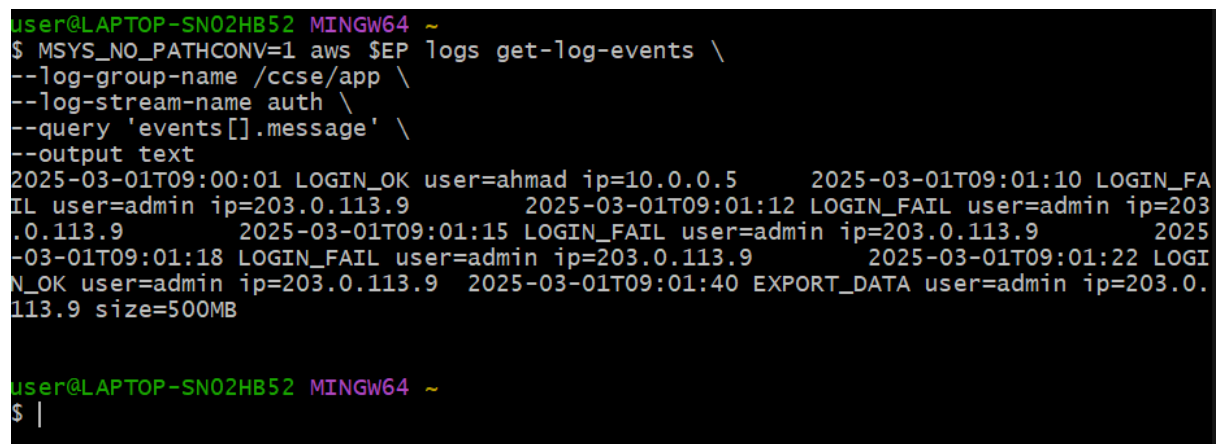
```
MSYS_NO_PATHCONV=1 aws $EP logs get-log-events \
 --log-group-name /ccse/app \
 --log-stream-name auth \
 --query 'events[].message' \
 --output text
'''
```

## Result

All seven application log events were successfully uploaded and retrieved from the `/ccse/app` log group and the `auth` log stream.

This confirmed that the application logs were successfully centralized using LocalStack CloudWatch Logs.

## Evidence

A terminal window screenshot showing a command being executed and its output. The command is: `MSYS_NO_PATHCONV=1 aws $EP logs get-log-events --log-group-name /ccse/app --log-stream-name auth --query 'events[].message' --output text`. The output displays seven log events in a table-like format, including timestamps, event names (LOGIN\_OK, LOGIN\_FAIL, EXPORT\_DATA), users (ahmad, admin), and IP addresses (10.0.0.5, 203.0.113.9).

```
user@LAPTOP-SN02HB52 MINGW64 ~
$ MSYS_NO_PATHCONV=1 aws $EP logs get-log-events \
--log-group-name /ccse/app \
--log-stream-name auth \
--query 'events[].message' \
--output text
2025-03-01T09:00:01 LOGIN_OK user=ahmad ip=10.0.0.5 2025-03-01T09:01:10 LOGIN_FA
IL user=admin ip=203.0.113.9 2025-03-01T09:01:12 LOGIN_FAIL user=admin ip=203
.0.113.9 2025-03-01T09:01:15 LOGIN_FAIL user=admin ip=203.0.113.9 2025
-03-01T09:01:18 LOGIN_FAIL user=admin ip=203.0.113.9 2025-03-01T09:01:22 LOGI
N_OK user=admin ip=203.0.113.9 2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.
113.9 size=500MB

user@LAPTOP-SN02HB52 MINGW64 ~
$ |
```

## Task 3 – Detect Repeated Failed Login Attempts

In this task, I analyzed the authentication log to identify repeated failed login attempts from the same IP address.

## Commands

```
'''bash
grep LOGIN_FAIL auth.log | awk '{print $4, $5}' | sort | uniq -c
'''
```

## Result

The command returned:

```
'''text
```

```
4 ip=203.0.113.9
```

```
...
```

This showed that the IP address `203.0.113.9` generated four failed login attempts.

This activity could indicate a possible brute-force login attempt.

## Evidence

```
user@LAPTOP-SN02HB52 MINGW64 ~
$ grep LOGIN_FAIL auth.log | awk '{print $4, $5}' | sort | uniq -c
4 ip=203.0.113.9

user@LAPTOP-SN02HB52 MINGW64 ~
$
```

## Task 4 – Tamper-Evident Hash-Chained Logs

In this task, I created a hash chain using SHA-256. Each log entry was combined with the previous hash before generating the next hash.

This made the logs tamper-evident because modifying a log entry changed the final hash value.

## Commands

```
```bash
```

```
PREV=0
```

```
while IFS= read -r line; do
```

```
    PREV=$(printf '%s%s' "$PREV" "$line" | sha256sum | cut -d' ' -f1)
```

```
    printf '%s | %s\n' "$line" "$PREV"
```

```
done < auth.log > auth.chain
```

```
cat auth.chain
```

```
```
```

## Result

A SHA-256 hash was successfully generated for every log entry.

The final hash of the original log chain was:

```
```text
```

```
ababa787b4bf524d9daddca8c48e4909fc105769a6f17574f42cefe8f81233cf
```

```
```
```

To test tampering, we changed the data export value from `500MB` to `5MB`.

## Commands

```
```bash
sed 's/500MB/5MB/' auth.log > auth.tampered

cat auth.tampered

PREV=0

while IFS= read -r line; do
    PREV=$(printf '%s%s' "$PREV" "$line" | sha256sum | cut -d' ' -f1)
done < auth.tampered

echo $PREV
```
```

## Result

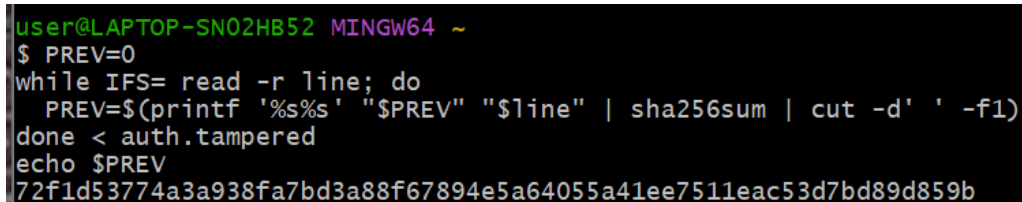
The final hash after tampering was:

```
```text
72f1d53774a3a938fa7bd3a88f67894e5a64055a41ee7511eac53d7bd89d859b
```
```

The original and tampered hash values were different.

This confirmed that the hash chain could detect changes made to the log file.

## Evidence



```
user@LAPTOP-SN02HB52 MINGW64 ~
$ PREV=0
while IFS= read -r line; do
 PREV=$(printf '%s%s' "$PREV" "$line" | sha256sum | cut -d' ' -f1)
done < auth.tampered
echo $PREV
72f1d53774a3a938fa7bd3a88f67894e5a64055a41ee7511eac53d7bd89d859b
```

## Task 5 – Detect an Incident Through Event Correlation

In this task, I correlated failed login attempts, a successful login, and a data export event from the same IP address.

The purpose was to identify whether the sequence of events represented a possible security incident.

## Commands

```
```bash
```

```
IP=203.0.113.9
```

```
FAILS=$(grep -c "LOGIN_FAIL.*$IP" auth.log)
```

```
SUCCESS=$(grep -c "LOGIN_OK.*$IP" auth.log)
```

```
EXPORT=$(grep -c "EXPORT_DATA.*$IP" auth.log)
```

```
echo "IP=$IP fails=$FAILS success=$SUCCESS export=$EXPORT"
```

```
if [ "$FAILS" -ge 3 ] && [ "$SUCCESS" -ge 1 ] && [ "$EXPORT" -ge 1 ]; then
```

```
    echo 'ALERT: probable brute-force -> compromise -> data exfiltration'
```

```
fi
```

```
...
```

Result

The output showed:

```
```text
```

```
IP=203.0.113.9 fails=4 success=1 export=1
```

```
ALERT: probable brute-force -> compromise -> data exfiltration
```

```
```
```

The system identified four failed login attempts, followed by one successful login and one data export event from the same IP address.

This sequence indicated a probable brute-force attack followed by account compromise and data exfiltration.

Evidence

```
user@LAPTOP-SN02HB52 MINGW64 ~
$ IP=203.0.113.9
FAILS=$(grep -c "LOGIN_FAIL.*$IP" auth.log)
SUCCESS=$(grep -c "LOGIN_OK.*$IP" auth.log)
EXPORT=$(grep -c "EXPORT_DATA.*$IP" auth.log)

echo "IP=$IP fails=$FAILS success=$SUCCESS export=$EXPORT"

if [ "$FAILS" -ge 3 ] && [ "$SUCCESS" -ge 1 ] && [ "$EXPORT" -ge 1 ]; then
    echo 'ALERT: probable brute-force -> compromise -> data exfiltration'
fi
IP=203.0.113.9 fails=4 success=1 export=1
ALERT: probable brute-force -> compromise -> data exfiltration

user@LAPTOP-SN02HB52 MINGW64 ~
$
```

Task 6 – Incident Containment & Evidence Collection

In this task, I blocked the suspicious IP address using an iptables rule. I also preserved the original application log as evidence and generated a SHA-256 hash to protect its integrity.

Commands

```
```bash
docker run --rm --cap-add=NET_ADMIN alpine sh -c \
'apk add -q iptables; iptables -A INPUT -s 203.0.113.9 -j DROP; iptables -L INPUT -n | tail -2'
```
```

Result

The firewall output showed:

```
```text
target prot opt source destination
DROP all -- 203.0.113.9 0.0.0.0/0
```
```

This confirmed that traffic from the suspicious IP address `203.0.113.9` was blocked.

The evidence file was then created and hashed.

Commands

```
```bash
cp auth.log evidence_$(date +%Y%m%d).log

sha256sum evidence_*.log > evidence.sha256

cat evidence.sha256
```
```

Result

The SHA-256 hash of the evidence file was:

```
```text
0adc5d2ac06cbbdd366099bcc0540c4c0f76946e71b52e4c99322731696a203b
*evidence_20260906.log
```
```

This SHA-256 hash was generated to support evidence integrity verification.

Evidence

```
user@LAPTOP-SN02HB52 MINGW64 ~  
$ cat evidence.sha256  
0adc5d2ac06cbbdd366099bcc0540c4c0f76946e71b52e4c99322731696a203b *evidence_2026090  
log  
  
user@LAPTOP-SN02HB52 MINGW64 ~  
$
```

Short Answer Questions

Question 1 – What is the difference between a log and an event? Give one example of each from this lab.

A log is a stored record that contains information about activities occurring in a system.

An event is one specific activity recorded inside the log.

For example, `auth.log` is the log file, while `LOGIN\_FAIL user=admin ip=203.0.113.9` is an individual security event recorded in the log.

Question 2 – Why must audit logs be tamper-proof? How does the hash chain used in this lab help?

Audit logs must be tamper-proof because they may be used as evidence during security investigations.

If an attacker changes or deletes log entries, the investigation may become unreliable.

The hash chain used in this lab makes changes detectable because each hash depends on the previous hash and the current log entry.

When the data export value was changed from `500MB` to `5MB`, the final hash changed. This proved that the log had been modified.

Question 3 – How did correlation detect the incident even though no single log line revealed the full attack?

Correlation combined multiple security events from the same IP address.

The log showed four failed login attempts, followed by one successful login and one data export event.

A single failed login or successful login may not prove an attack. However, when these events were analyzed together, they showed a pattern of:

`Brute-force attempt -> Account compromise -> Data exfiltration`

This allowed the incident to be detected.

Question 4 – What incident-response steps were performed in this lab, and what was the goal of each?

The following incident-response steps were performed:

- Detection: Suspicious login and data export activities were identified.
- Analysis: Related events from the same IP address were correlated to understand the incident.
- Containment: The suspicious IP address was blocked using an iptables DROP rule.
- Evidence Collection: The original application log was copied and preserved as evidence.
- Integrity Verification: A SHA-256 hash was generated and verified to confirm that the evidence had not been modified.

Question 5 – How can the same logs support both security monitoring and compliance evidence?

Logs support security monitoring by helping analysts identify suspicious behavior such as repeated failed login attempts, unusual successful logins, and data export activities.

The same logs can also support compliance because they provide records of system and security activities.

The records can show what happened, when it happened, and what security actions were taken.

Using SHA-256 hashes also helps demonstrate that the evidence has maintained its integrity.

Verification Command

The final verification confirmed that the centralized log group was available and that the collected evidence had not been modified.

Commands

```
```bash
sha256sum -c evidence.sha256
```

```
MSYS_NO_PATHCONV=1 aws $EP logs describe-log-groups
```
```

Result

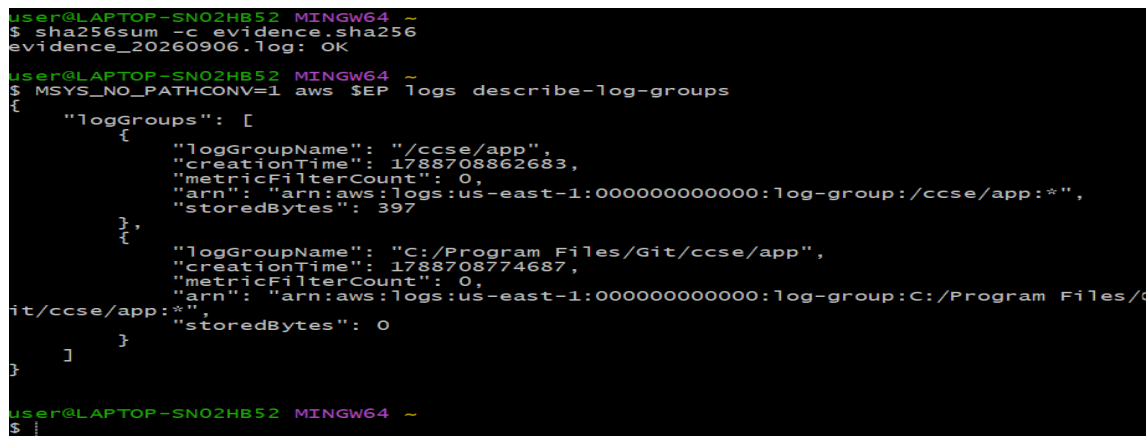
The evidence integrity verification returned:

```
```text
evidence_20260906.log: OK
```
```

This confirmed that the evidence file had not been modified after the SHA-256 hash was generated.

The LocalStack verification also confirmed that the correct `/ccse/app` log group existed and contained stored log data.

Evidence



```
user@LAPTOP-SN02HB52 MINGW64 ~
$ sha256sum -c evidence.sha256
evidence_20260906.log: OK

user@LAPTOP-SN02HB52 MINGW64 ~
$ MSYS_NO_PATHCONV=1 aws s3 logs describe-log-groups
{
  "logGroups": [
    {
      "logGroupName": "/ccse/app",
      "creationTime": 1788708862683,
      "metricFilterCount": 0,
      "arn": "arn:aws:logs:us-east-1:000000000000:log-group:/ccse/app:*",
      "storedBytes": 397
    },
    {
      "logGroupName": "C:/Program Files/Git/ccse/app",
      "creationTime": 1788708774687,
      "metricFilterCount": 0,
      "arn": "arn:aws:logs:us-east-1:000000000000:log-group:C:/Program Files/Git/ccse/app:*",
      "storedBytes": 0
    }
  ]
}
```

Cleanup / Tear Down

After completing the lab, collecting the required evidence, and performing the verification, I removed the temporary files and the LocalStack container.

Commands

```
```bash
rm -f auth.log auth.chain auth.tampered evidence_*.log evidence.sha256

docker stop localstack && docker rm localstack
```
```

Result

The temporary lab files were removed, and the LocalStack container was stopped and deleted.

```
user@LAPTOP-SN02HB52 MINGW64 ~  
$ rm -f auth.log auth.chain auth.tampered evidence_*.log evidence.sha256  
  
user@LAPTOP-SN02HB52 MINGW64 ~  
$ docker stop localstack && docker rm localstack  
localstack  
localstack  
  
user@LAPTOP-SN02HB52 MINGW64 ~  
$
```

Security Best-Practices Checklist

- ☒ Logs are centralized, not left scattered on each host.
- ☒ Security-relevant activity (failed logins) can be queried.
- ☒ Logs are tamper-evident (hash chain) and forwarded to a separate store.
- ☒ An incident is detected by correlating multiple events.
- ☒ Incident response performed: contain, collect evidence, document.